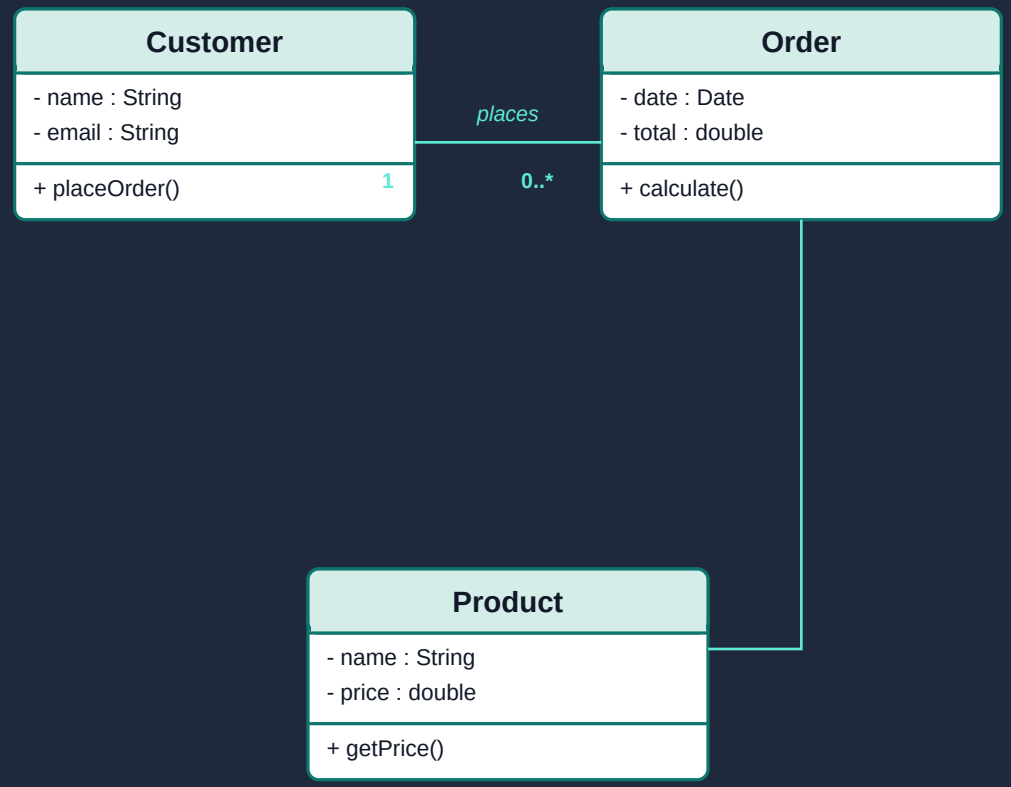
UML | SOFTWARE ENGINEERING

# Class Diagram

The blueprint of a system: its classes, their data, and how they are connected.




Classes | Relationships | Multiplicity

**Prepared by Amira Mahmoud**

# What is a Class Diagram?

- A UML diagram that shows the structure of a system: its classes and how they are related.
- It answers two questions: what is the system made of, and who is connected to whom?
- It is a static view. There is no time and no order of steps.
- It is the map that programmers follow to write code (Java, C#, Python...).

Think of it as

The architect's blueprint of a house: it shows the rooms and the doors, not the people walking inside.

### Use Case Diagram

WHAT the system does for its users.

*Actors and goals.*

### Class Diagram

WHAT the system is made of.

*Classes and relations.*

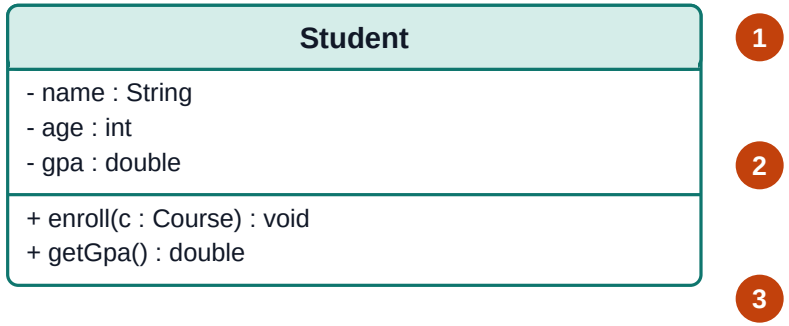
**YOU ARE HERE**

### Sequence Diagram

HOW the parts talk, step by step.

*Messages over time.*

# Anatomy of a Class




1 **Class name**
Top box. A singular noun that starts with a capital letter: Student, Order, Book.

2 **Attributes**
Middle box. The data the class keeps: name, age, gpa.

3 **Methods**
Bottom box. What the class can do: enroll, getGpa. Also called operations.

Class = the blueprint. Object = one real thing built from it, like Ali the student.

Tip: write each class as a rectangle with 3 compartments. Empty compartments can be left empty.

# Visibility and Member Syntax

**+** **Public**
Any class can use it.

**-** **Private**
Only inside the same class.

**#** **Protected**
The class and its subclasses.

**~** **Package**
Classes in the same package.

## Attribute

```
visibility name : type = default
```

```
- balance : double = 0
```

## Method

```
visibility name(param : type) : returnType
```

```
+ deposit(amount : double) : void
```

## Special notation

| | |
|---|---|
| Static member | **count : int** |
| Abstract class / method | ***Shape*** |
| Derived attribute | **/ age : int** |
| Read-only | **id : int {readOnly}** |

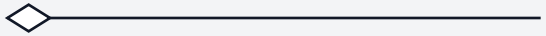
# Relationships at a Glance



## Association

**Knows / uses**

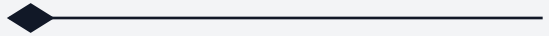
A solid line. One class is linked to another.



## Aggregation

**Has-a (weak)**

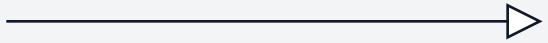
Hollow diamond. The part can live alone.



## Composition

**Owns (strong)**

Filled diamond. The part dies with the whole.



## Inheritance

**Is-a**

Hollow triangle pointing to the parent.

## Realization

**Implements**

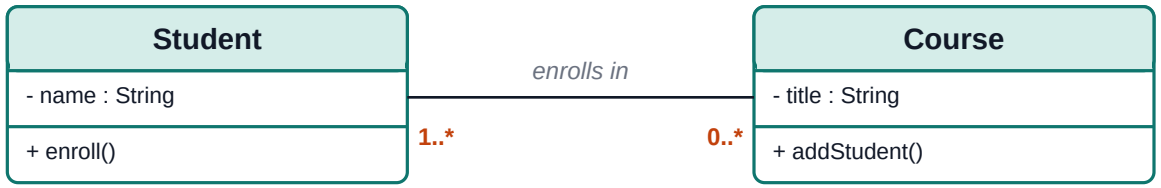
Dashed line, hollow triangle to the interface.

## Dependency

**Uses briefly**

Dashed line with an open arrow.

# Association and Multiplicity




Read it aloud: a Student enrolls in zero or many Courses. A Course has one or many Students.

A Customer places zero or many Orders. Every Order belongs to exactly one Customer.

## Multiplicity

| | |
|---|---|
| `1` | Exactly one |
| `0..1` | Zero or one (optional) |
| `* or 0..*` | Zero or many |
| `1..*` | One or many |
| `2..5` | From two to five |

# Aggregation vs Composition

## Aggregation

**Has-a (weak)**

A Player can exist without the Team.

Diamond: hollow. It sits next to the whole (Team).

## Composition

**Owns (strong)**

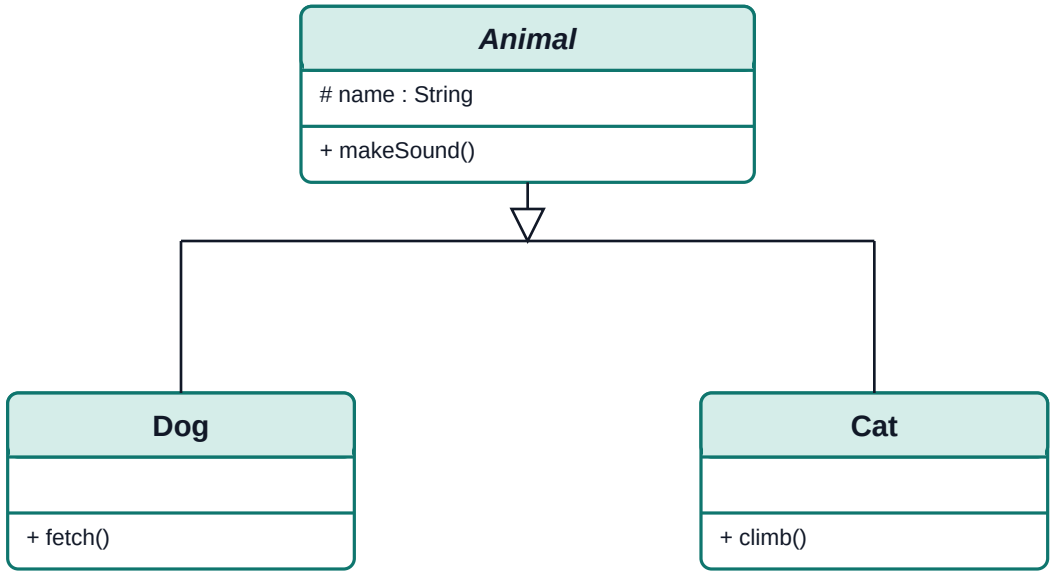
A Room is destroyed with the House.

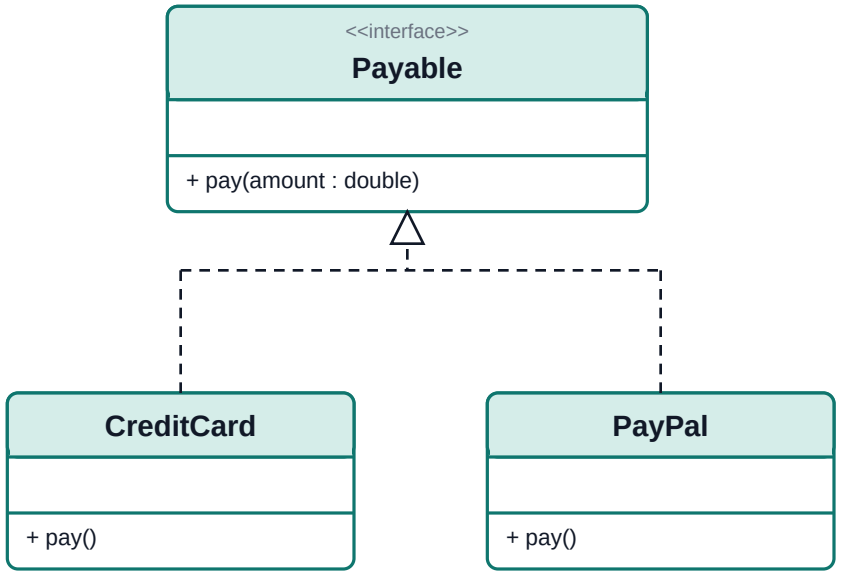
Diamond: filled. It sits next to the whole (House).

The test

**Delete the whole. Does the part survive?  Yes = Aggregation.  No = Composition.**

# Inheritance and Realization



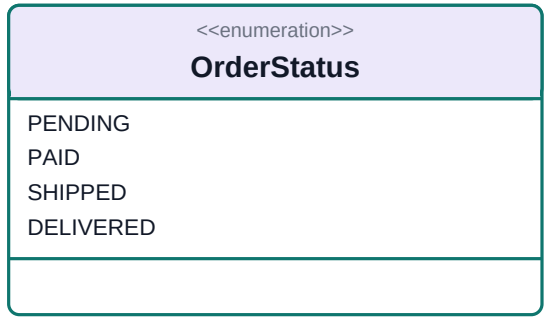

### Inheritance = is-a

A Dog is an Animal. It gets name and makeSound() from the parent. Solid line, hollow triangle at the parent. The italic name means abstract.

### Realization = implements

CreditCard and PayPal promise to provide pay(). Dashed line, hollow triangle at the interface.

# Dependency and Enumeration




## Dependency

One class uses another only for a short time: as a method parameter, a local variable or a return type. It is not stored as an attribute.

**Stored as an attribute: association (solid). Used briefly: dependency (dashed).**

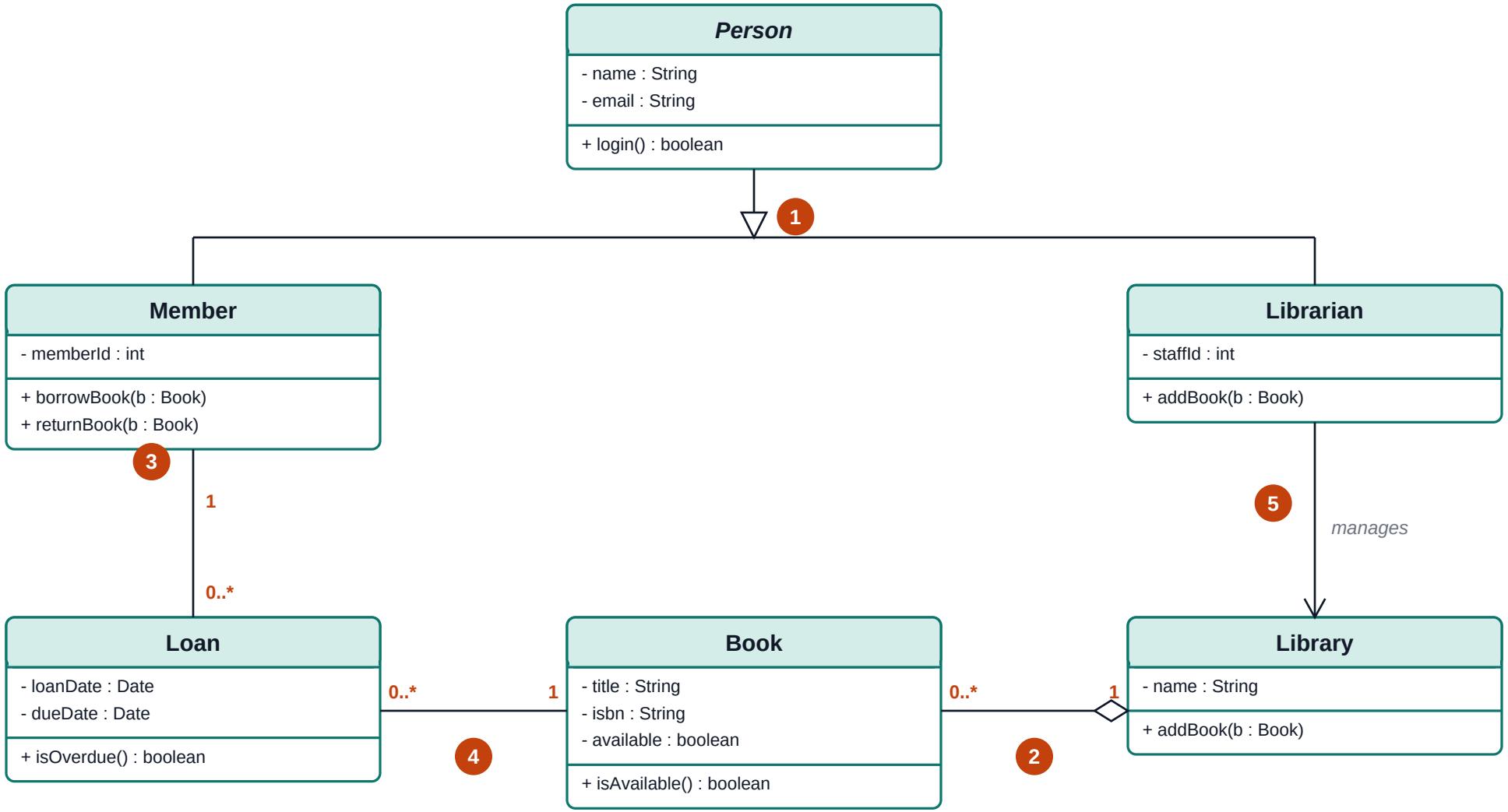
## Enumeration

A fixed list of values. Use it for a status, a day or a type, so nobody can type a wrong value.

Which arrow?

**Stored = solid line.  Used briefly = dashed line.  Is-a = hollow triangle.**

# Example: Online Library



# Walk Through the Example

1. Member and Librarian inherit from Person. They share name, email and login().
2. A Library has many Books. Aggregation: a Book can exist without the Library.
3. A Member can have zero or many Loans. Each Loan belongs to one Member.
4. Each Loan is for exactly one Book. A Book can appear in many Loans over time.
5. A Librarian manages the Library. The arrow shows who knows whom.

### Link to Sequence Diagram

The methods in the classes, like borrowBook() and isAvailable(), become the messages in the sequence diagram.

**Class diagram = the parts. Sequence diagram = the conversation.**

# 6 Steps to Draw One

1. **Pick the system.**
   Example: an online library.

2. **Find the classes.**
   Nouns in the story: Member, Book, Loan.

3. **Add attributes.**
   The data each class keeps.

4. **Add methods.**
   Verbs: what each class can do.

5. **Connect the classes.**
   Association, inheritance, aggregation...

6. **Add multiplicity and check.**
   1, 0..1, 0..* on both ends of every line.

**Nouns become classes.  Verbs become methods.**

# Common Mistakes

| | |
|---|---|
| ✗ The diamond is on the part. | ✓ Put the diamond next to the whole. |
| ✗ The inheritance triangle points to the child. | ✓ The triangle always points to the parent. |
| ✗ Car extends Engine (a car is not an engine). | ✓ Use composition for has-a. Inherit only for is-a. |
| ✗ No multiplicity on the lines. | ✓ Write 1, 0..1 or 0..* at both ends. |
| ✗ Verbs as class names: Borrow. | ✓ Singular nouns: Loan, Book, Member. |
| ✗ Every tiny detail in every class. | ✓ Show only the important attributes and methods. |

# Your Turn: Online Shop

## The scenario

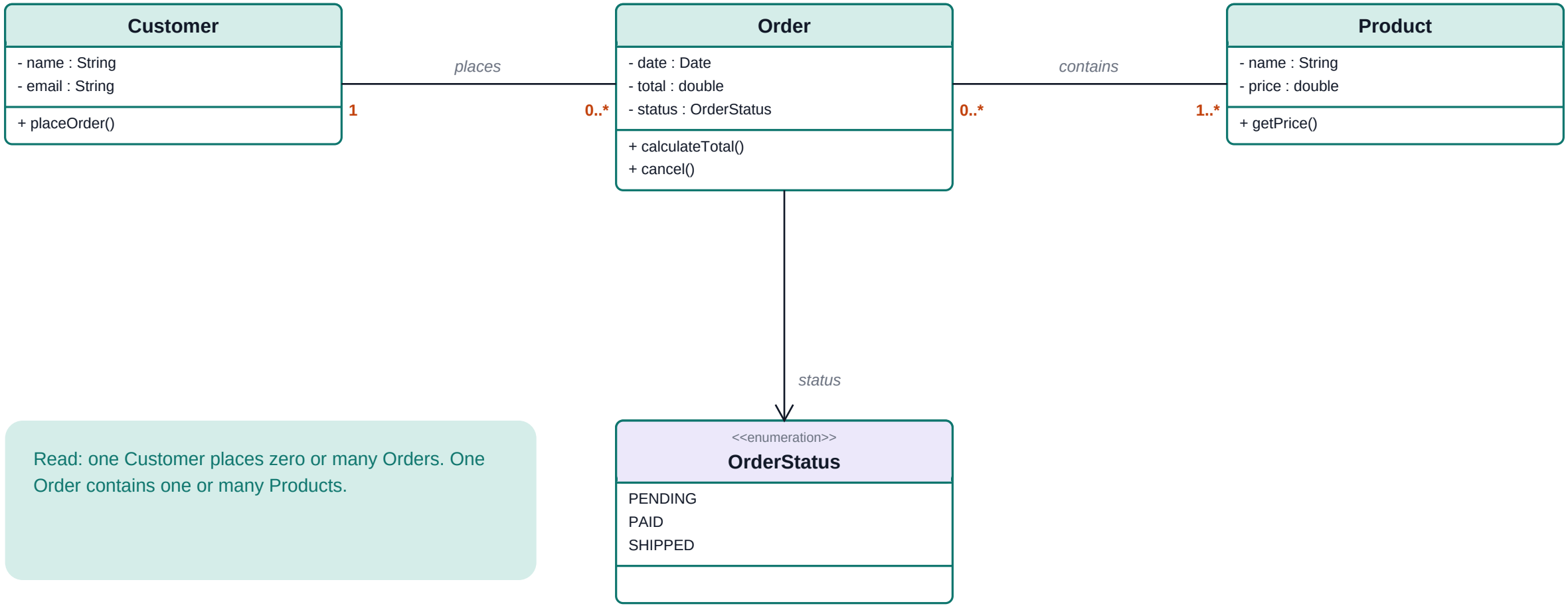
A Customer places many Orders. Each Order contains one or more Products. An Order has a date, a total and a status: PENDING, PAID or SHIPPED. A Product has a name and a price.

## Draw the class diagram

Classes: Customer, Order, Product, OrderStatus. Add 2 attributes and 1 method to each class. Show multiplicity on every line. Time: 7 minutes.

1. **Underline the nouns.**
2. **Status is an enumeration.**
3. **Every line needs multiplicity.**

# Answer: Online Shop




Read: one Customer places zero or many Orders. One Order contains one or many Products.

# Summary

- ✓ A class diagram shows what the system is made of, not the order of steps.
- ✓ A class has 3 boxes: name, attributes, methods. Visibility: + - # ~
- ✓ Association, aggregation, composition, inheritance, realization, dependency.
- ✓ Multiplicity (1, 0..1, 0..*, 1..*) says how many objects are linked.
- ✓ Nouns become classes, verbs become methods. The triangle points to the parent.

## Questions?

**Prepared by Amira Mahmoud**

Software Engineering Section